

π -games

Normal-Form and Extensive-Form Games in Typst

Piotr Kuszewski June 2026

1 Introduction

The `pi-games` bundle provides Typst tools for typesetting both representations of a game used in game-theory courses and research:

Normal-form (strategic-form) games payoff matrices, drawn by `pi-game-normal.typ`.

Extensive-form games sequential game trees, drawn by `pi-game-trees.typ`.

Both libraries share a single colour palette (`pi-game-palette.typ`) so that player colours are consistent across the two representations. The umbrella module `pi-games.typ` re-exports everything, so a single import exposes the full API:

```
1 #import "@preview/cetz:0.5.2" as cetz
2 #import "pi-games.typ": *
```

 Typst

Extensive-form trees are drawn inside a `cetz.canvas` block, so `cetz` is imported explicitly as well. The manual first covers normal-form games, then extensive-form trees, and a combined API reference appears at the end.

1.1 Normal-Form Functions

game-normal-form draws a two-player $N \times M$ payoff matrix. Each cell displays both players' payoffs separated by a comma, coloured by player.

game-three-player-normal-form draws a three-player game as a collection of $N \times M$ matrices — one per strategy of Player 3 — laid out in rows.

Both functions measure content at layout time and automatically size each cell to fit its widest payoff expression or strategy label, so the typesetter never needs to specify column widths by hand.

1.2 Extensive-Form Functions

The `pi-game-trees` library covers the full vocabulary of extensive-form games:

- **game-node**: a decision node for a named player, drawn as a filled or outlined circle in that player's colour.
- **game-nature**: a Nature / chance node, drawn in neutral grey.
- **game-terminal**: a terminal (leaf) node with a coloured payoff vector.
- **game-branch**: a directed edge with an optional action label, placed either on the branch line or to its east or west side.

- `game-prob`: convenience wrapper around `game-branch` that renders the label in Nature’s grey — intended for probability annotations.
- `game-info`set: an information set connecting two or more decision nodes, drawn either as a dashed line or as a rounded bracket ribbon.
- `game-subgame`: a proper-subgame triangle marker (Osborne–Rubinstein style).
- `game-highlight`: a bold coloured overlay for marking equilibrium paths.
- `game-payoffs`, `game-player`, `game-player-default`: inline text helpers for body text and captions.

Every tree is drawn inside a `cetz.canvas` block:

```

1 #figure(
2   cetz.canvas({
3     import cetz.draw: *
4     // game-... calls go here
5   }),
6   caption: [My game tree],
7 )

```

Player colours are shown below. They can be overridden by redefining `game-pal` after import (see Section 11).

Player 1
Player 2
Player 3
Player 4
Player 5
Nature

1.3 Coordinate System

CeTZ uses a standard mathematical coordinate system: the x -axis points right and the y -axis points *up*. Extensive-form game trees conventionally grow *downward*, so the root node is placed at $y = 0$ and each successive level has a more negative y -coordinate. A typical vertical tree uses level spacing $\Delta y \approx -2$ cm and sibling spacing $\Delta x \approx 1.5$ –3 cm per level.

The order of calls within a `cetz.canvas` block determines the drawing order (painter’s algorithm). Draw branches and information sets *before* the nodes so that node circles appear on top of the line endpoints. Place `game-highlight` calls between the branches and the nodes for the same reason.

Normal-Form Games

2 game-normal-form

2.1 Payoff Array Structure

The `payoffs` argument is a 2-dimensional array. Element `payoffs.at(r).at(c)` holds a 2-element array (`v1`, `v2`) where `v1` is Player 1's payoff and `v2` is Player 2's payoff when Player 1 plays row `r` and Player 2 plays column `c`. Both values are ordinary Typst content and may contain arbitrary markup or mathematics.

```
1  payoffs: (tTypst
2    // row 0
3    ( (v1_00, v2_00), (v1_01, v2_01) ),
4    // row 1
5    ( (v1_10, v2_10), (v1_11, v2_11) ),
6  )
```

2.2 Auto-Sizing

Cell width is determined automatically at render time as the maximum of three quantities:

1. **cell-width**: the user-supplied lower bound (defaults to `game-cell-width`, initially 5em).
2. **Strategy-label width**: the rendered width of the widest Player 2 strategy label plus 2em of horizontal padding. This ensures each column is never narrower than its header.
3. **Payoff width**: the rendered width of the widest payoff pair `v1`, `v2` — including best-response underlines where active — plus 2em of padding.

The measurement happens inside a `context` block, so it uses the exact font metrics of every piece of content, including mathematical formulas of arbitrary complexity. The **left margin** (Player 1 strategy labels and rotated player name) is auto-sized to the widest Player 1 label; the **top margin** (Player 2 strategy labels and player name) is auto-sized to the tallest Player 2 label. **Cell height** is fixed at `cell-height` (defaults to `game-cell-height`, initially 2em) and is not auto-sized, since all payoff content is placed on a single line.

2.3 Best Responses and Nash Equilibria

- **p1-best** / **p2-best**: arrays of (`row`, `col`) tuples. In each listed cell the respective player's payoff is underlined with a 1 pt stroke in their colour (`game-pal.at(0)` and `game-pal.at(1)`).

- **nash**: array of (row, col) tuples. A coloured rectangle in `game-nash-color` is drawn just inside the cell border.

Both underlines and Nash rectangles are taken into account when auto-sizing, so highlighting never causes payoffs to overflow their cells.

2.4 Examples

2.4.1 Prisoner's Dilemma

The Prisoner's Dilemma is the textbook example of a dominant-strategy equilibrium. Defection strictly dominates Cooperation for both players, so (D,D) is the unique Nash equilibrium even though (C,C) Pareto-dominates it.

```

1  #game-normal-form(
2    [Prisoner 1], [Prisoner 2],
3    ([C], [D]),
4    ([C], [D]),
5    (
6      ([[3$], [3$]], [[0$], [5$]]),
7      ([[5$], [0$]], [[1$], [1$]]),
8    ),
9    p1-best: ((1, 0), (1, 1)),
10   p2-best: ((0, 1), (1, 1)),
11   nash: ((1, 1)),
12 )

```

		Prisoner 2	
		C	D
Prisoner 1	C	3, 3	0, 5
	D	5, 0	1, 1

2.4.2 Battle of Sexes

In the Battle of Sexes the players want to coordinate but disagree on which outcome to coordinate on. There are two pure-strategy Nash equilibria — both choose Opera, or both choose Football — and a mixed-strategy equilibrium in between.

```

1  #game-normal-form(
2    [She], [He],
3    ([Opera], [Football]),
4    ([Opera], [Football]),
5    (
6      ([[2$], [1$]], [[0$], [0$]]),
7      ([[0$], [0$]], [[1$], [2$]]),
8    ),

```

```

9   p1-best: ((0, 0), (1, 1)),
10  p2-best: ((0, 0), (1, 1)),
11  nash: ((0, 0), (1, 1)),
12 )

```

		He	
		Opera	Football
She	Opera	2, 1	0, 0
	Football	0, 0	1, 2

2.4.3 Mixed-Strategy Equilibrium: Matching Pennies

Strategy labels are arbitrary Typst content, so mixed-strategy equilibrium probabilities can be embedded directly in the label. Matching Pennies has no pure-strategy Nash equilibrium; the unique Nash equilibrium requires each player to play Heads with probability $p^* = q^* = \frac{1}{2}$. Including $[p]$ and $[q]$ in the strategy names makes the equilibrium condition explicit. The column width auto-adjusts to accommodate the wider labels.

```

1  #game-normal-form(
2    [P1], [P2],
3    ([H #h(1mm) $[p]$, [T #h(1mm) $[1-p]$\]),
4    ([H $[q]$, [T $[1-q]$\]),
5    (
6      (([$1$], [$-1$]), ([$-1$], [$1$])),
7      (([$-1$], [$1$]), ([$1$], [$-1$])),
8    ),
9  )

```

		P2	
		$[q]$ H	$[1-q]$ T
P1	$[p]$ H	1, -1	-1, 1
	$[1-p]$ T	-1, 1	1, -1

2.4.4 Parametric Payoffs: 3×3 Coordination Game

Payoff values may contain mathematical variables and expressions of any complexity. Auto-sizing measures the actual rendered width of each expression to determine the cell size. The symmetric 3×3 coordination game has payoff a when players match and b otherwise; all three pure-strategy Nash equilibria (A,A) , (B,B) , (C,C) exist for any $a > b$.

```

1  #game-normal-form(
2    [P1], [P2],

```

```

3    ([A], [B], [C]),
4    ([A], [B], [C]),
5    (
6        ([[a$], [a$]], [[b$], [b$]], [[b$], [b$]]),
7        ([[b$], [b$]], [[a$], [a$]], [[b$], [b$]]),
8        ([[b$], [b$]], [[b$], [b$]], [[a$], [a$]]),
9    ),
10   p1-best: ((0, 0), (1, 1), (2, 2)),
11   p2-best: ((0, 0), (1, 1), (2, 2)),
12   nash:    ((0, 0), (1, 1), (2, 2)),
13 )

```

		P2		
		A	B	C
P1	A	a, a	b, b	b, b
	B	b, b	a, a	b, b
	C	b, b	b, b	a, a

3 game-three-player-normal-form

3.1 Payoff Array Structure

The `payoffs` argument is a 3-dimensional array. Element `payoffs.at(k).at(r).at(c)` holds a 3-element array (`v1`, `v2`, `v3`):

k (outermost) index of Player 3's strategy.

r index of Player 1's strategy (row).

c index of Player 2's strategy (column).

```

1  payoffs: (
2      // k = 0  (first Player 3 strategy)
3      (
4          ((v1, v2, v3), (v1, v2, v3)),  // r = 0
5          ((v1, v2, v3), (v1, v2, v3)),  // r = 1
6      ),
7      // k = 1  (second Player 3 strategy)
8      (
9          ((v1, v2, v3), (v1, v2, v3)),
10         ((v1, v2, v3), (v1, v2, v3)),
11     ),
12 )

```

 Typst

Best-response and Nash coordinates use 3-element tuples (`k`, `row`, `col`) throughout.

3.2 Layout and Auto-Sizing

Sub-matrices are placed left to right in rows of at most `games-per-row` sub-matrices (default 2). If Player 3 has more strategies than fit in a row, additional rows are added below. A lone sub-matrix in the last row is horizontally centred within the full row width.

All sub-matrices share identical cell dimensions, computed once from the entire payoff array: the function scans every payoff triple in every cell and every sub-matrix to find the widest rendered content, then combines that with the widest Player 2 strategy label to determine cell width.

Above each sub-matrix (from bottom to top): Player 2's strategy labels, then Player 2's name in bold, then Player 3's strategy label in bold. Player 1's name and strategy labels appear once per row of sub-matrices, to the left. Player 3's player name appears once above the first row of sub-matrices.

3.3 Example: Three-Player Stag Hunt

All three players must cooperate to catch the stag (payoff 3). A player who deviates and hunts the hare alone gets 1; players who pursue the stag without all partners cooperating get 0. Hunting hare is always safe, so (H,H,H) is always a Nash equilibrium; (S,S,S) is the other.

```
1  #game-three-player-normal-form( tTypst
2    [P1], [P2], [P3],
3    ([S], [H]),
4    ([S], [H]),
5    ([S], [H]),
6    (
7      // P3 plays S
8      (
9        ([[3],[3],[3]], ([0],[1],[0])),
10       ([[1],[0],[0]], ([1],[1],[0])),
11      ),
12     // P3 plays H
13     (
14       ([[0],[0],[1]], ([0],[1],[1])),
15       ([[1],[0],[1]], ([1],[1],[1])),
16     ),
17   ),
18   nash:    ((0,0,0), (1,1,1)),
19   p1-best: ((0,0,0), (0,1,1), (1,1,0), (1,1,1)),
20   p2-best: ((0,0,0), (0,1,1), (1,0,1), (1,1,1)),
21   p3-best: ((0,0,0), (1,0,1), (1,1,0), (1,1,1)),
22 )
```


Extensive-Form Game Trees

5 game-node — Decision Nodes

A decision node is a circle in the player's colour. By default it is solid-filled (`style: "filled"`). Pass `style: "open"` for a hollow circle or `style: "dot"` for a smaller disc suitable for compact trees.

The `label` parameter accepts arbitrary Typst content (plain text, math, formatted content). The direction `la` controls which side of the node the label appears on. The eight compass directions `"n"`, `"ne"`, `"e"`, `"se"`, `"s"`, `"sw"`, `"w"`, `"nw"` are all supported. The default is `"n"` (label above the node).

Assign a `name` when the node coordinate must be referenced later — for example, to connect nodes in `game-info`set.

```
1 #cetz.canvas({
2   import cetz.draw: *
3   game-node((0, 0),
4     player: 1,
5     label: [Player 1],
6     la: "n")
7   game-node((2, 0),
8     player: 2,
9     label: [Player 2],
10    la: "n",
11    style: "open")
12   game-node((4, 0),
13     player: 3,
14     label: [P3],
15     la: "n",
16     style: "dot")
17 })
```

Player 1 Player 2 P3



5.1 game-nature — Nature / Chance Nodes

A Nature node is drawn identically to `game-node` but always in `game-nature-color` (grey). The default label is `N`; any content may be substituted.

```
1 #cetz.canvas({
2   import cetz.draw: *
```

```

3  game-nature((0, 0), label: [$cal(N)$], la: "n")
4  })

```



5.2 game-terminal — Terminal Nodes

A terminal node is a small filled dot (radius `game-terminal-radius`) followed by a payoff vector. Each player's payoff is coloured with that player's colour. The direction `la` places the vector below ("s", default), above, or to either side of the dot. Pass `parens: false` to suppress parentheses.

```

1  #cetz.canvas({
2      import cetz.draw: *
3      game-terminal((0, 0),
4          payoffs: ([2], [1]),
5          la: "s")
6      game-terminal((2, 0),
7          payoffs: ([$a$], [$b$]),
8          la: "s",
9          parens: false)
10     game-terminal((4, 0),
11         payoffs: ([$-1$], [0], [1]),
12         la: "s")
13 })

```



6 game-branch — Branches

A branch is a straight line from `from` to `to`. The optional `action` label is placed at the fractional position `apos` along the branch (default 0.5 = midpoint). The `arrow` parameter adds a CeTZ arrowhead at the child end.

6.1 Label Placement: **side**

The `side` parameter controls how the action label is positioned:

- `side: none` (default): the label sits directly on the branch line, covered by a white rectangle so the line does not bleed through the text.
- `side: "e"`: the label is offset to the **east** (absolute right) side of the branch by `act` centimetres, anchored at its west edge.
- `side: "w"`: the label is offset to the **west** (absolute left) side of the branch by `act` centimetres, anchored at its east edge.

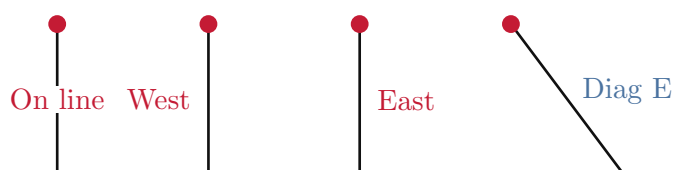
East and west are defined absolutely (positive and negative x) regardless of the slope of the branch. The correct 90° perpendicular is computed automatically from the branch

direction vector. Using "west" and "east" CeTZ anchors ensures that labels with different capital heights are baseline-consistent across branches.

```

1  #cetz.canvas({
2      import cetz.draw: *
3      // Label on the line
4      game-branch((0, 0), (0, -2),
5          action: [On line],
6          player: 1)
7      // West side
8      game-branch((2, 0), (2, -2),
9          action: [West],
10         player: 1,
11         side: "w")
12     // East side
13     game-branch((4, 0), (4, -2),
14         action: [East],
15         player: 1,
16         side: "e")
17     // Diagonal branch with side "e"
18     game-branch((6, 0), (7.5, -2),
19         action: [Diag E],
20         player: 2,
21         side: "e")
22     // Nodes drawn last so they appear on top of branch endpoints
23     for x in (0, 2, 4, 6) { game-node((x, 0), player: 1) }
24 })

```



6.2 Label Position: apos and act

apos moves the label along the branch: 0.3 places it closer to the parent, 0.7 closer to the child. act controls the perpendicular distance (in cm) from the branch line to the label anchor — increase it when labels would otherwise overlap a nearby branch.

6.3 game-prob — Probability Labels

game-prob is a thin wrapper around game-branch that sets player: 0, rendering the action label in game-nature-color. All other parameters are identical to game-branch. Use it to annotate Nature branches with probabilities.

```

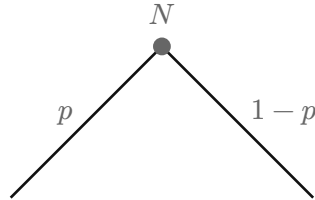
1  #cetz.canvas({

```

```

2  import cetz.draw: *
3  game-prob((0, 0), (-2, -2), action: [$p$], side: "w")
4  game-prob((0, 0), ( 2, -2), action: [$1 - p$], side: "e")
5  game-nature((0, 0), la: "n")
6  })

```



7 game-infoset — Information Sets

An information set connects two or more decision nodes to indicate that the player at those nodes cannot distinguish between them. Two call forms are accepted:

Two-point form `game-infoset(pos1, pos2, ...)` — two separate positional arguments.

List form `game-infoset((pos1, pos2, pos3, ...), ...)` — a single argument that is an array of positions. Draws segments $p_1 \rightarrow p_2, p_2 \rightarrow p_3, \dots$ in order.

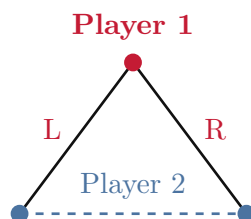
7.1 Dashed Style

The default style: "dashed" draws dashed line segments between consecutive points. An optional label is placed at the midpoint of the span; `la` controls its direction.

```

1  #cetz.canvas({
2    import cetz.draw: *
3    game-branch(( 0, 0), (-1.5, -2), action: [L], player: 1, side: "w")
4    game-branch(( 0, 0), ( 1.5, -2), action: [R], player: 1, side: "e")
5    game-infoset((-1.5, -2), (1.5, -2),
6      player: 2,
7      label: [Player 2],
8      la: "n")
9    game-node((0, 0), player: 1, label: [Player 1], la: "n")
10   game-node((-1.5, -2), player: 2)
11   game-node(( 1.5, -2), player: 2)
12 })

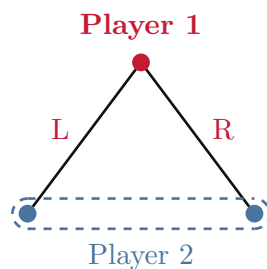
```



7.2 Bracket Style

style: "bracket" draws a rounded bracket ribbon. Two dashed rails run `game-tick` cm to each side of the spine between the given points; each endpoint is closed by an outward-facing half-circle arc. The bracket visually groups the nodes into a single decision region without crossing any branches.

```
1  #cetz.canvas({
2      import cetz.draw: *
3      game-branch(( 0, 0), (-1.5, -2), action: [L], player: 1, side: "w")
4      game-branch(( 0, 0), ( 1.5, -2), action: [R], player: 1, side: "e")
5      game-infoset((-1.5, -2), (1.5, -2),
6          player: 2,
7          style: "bracket",
8          label: [Player 2],
9          la: "s")
10     game-node((0, 0), player: 1, label: [Player 1], la: "n")
11     game-node((-1.5, -2), player: 2)
12     game-node(( 1.5, -2), player: 2)
13 })
```



7.3 Curvature: angle

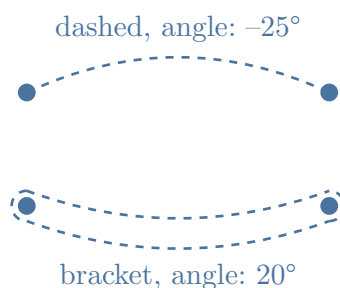
Any segment can be bent with the `angle` parameter. A positive angle curves the segment toward its CCW perpendicular (away from the tree's branches for typical horizontal information sets). The Bézier control point is displaced from the segment midpoint by $\text{chord} \times \tan(\text{angle})/2$. For "bracket" style both rails bend by the same angle. An array of angles applies one per segment.

```
1  #cetz.canvas({
2      import cetz.draw: *
3      // Dashed, bent downward (angle < 0 for southward bow)
4      game-infoset((-2, 0), (2, 0),
5          player: 2,
6          style: "dashed",
7          label: [dashed, angle: -25°],
8          la: "n",
9          angle: -25deg)
```

```

10 // Bracket, bent upward
11 game-infoset((-2, -1.5), (2, -1.5),
12     player: 2,
13     style: "bracket",
14     label: [bracket, angle: 20°],
15     la: "s",
16     angle: 20deg)
17 for x in (-2, 2) {
18     game-node((x, 0), player: 2)
19     game-node((x, -1.5), player: 2)
20 }
21 })

```



7.4 Multi-Point Information Sets

Pass an array of three or more positions as the first argument to span a broken-line information set. Each intermediate rail point uses the bisector of its two adjacent segment perpendiculars, so the rails meet cleanly at any bend angle.

```

1 #cetz.canvas({
2     import cetz.draw: *
3     game-infoset(
4         ((-3, 0), (0, 0), (3, 0)),
5         player: 2, style: "bracket", label: [three nodes], la: "s",
6     )
7     for x in (-3, 0, 3) { game-node((x, 0), player: 2) }
8 })

```



8 Complete Examples

8.1 Entry Deterrence (Sequential, Perfect Information)

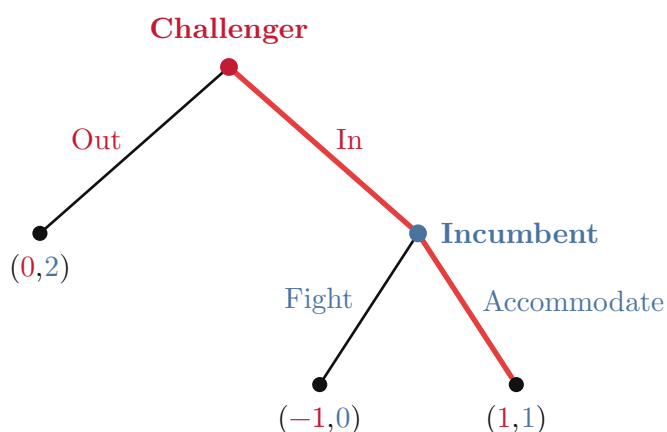
A **Challenger** decides whether to enter a market. If she enters, the **Incumbent** either *fights* or *accommodates*. The unique subgame-perfect Nash equilibrium (SPNE) survives

backward induction: the Challenger enters, the Incumbent accommodates, yielding payoffs $(1, 1)$. The bold path shows the SPNE.

```

1  #cetz.canvas({
2      import cetz.draw: *
3      game-branch((0,0), (-2.5,-2.2), action: [Out], player: 1, side: "w")
4      game-terminal((-2.5,-2.2), payoffs: ([0], [2]))
5      game-branch((0,0), ( 2.5,-2.2), action: [In], player: 1, side: "e")
6      game-branch((2.5,-2.2), (1.2,-4.2), action: [Fight], player: 2, side:
7          "w")
8      game-terminal((1.2,-4.2), payoffs: ([-$1$], [0]))
9      game-branch((2.5,-2.2), (3.8,-4.2), action: [Accommodate], player: 2,
10         side: "e")
11     game-terminal((3.8,-4.2), payoffs: ([1], [1]))
12     game-highlight((0,0), (2.5,-2.2))
13     game-highlight((2.5,-2.2),(3.8,-4.2))
14     game-node((0,0), player: 1, label: [Challenger], la: "n")
15     game-node((2.5,-2.2), player: 2, label: [Incumbent], la: "e")
16 })

```



8.2 Battle of the Sexes (Simultaneous Moves)

Player 1 moves first in the tree, but **Player 2** does not observe the move — the two **Player 2** nodes therefore belong to the same information set (dashed line). This makes the game strategically equivalent to its normal form. There are two pure-strategy Nash equilibria: $(2, 1)$ and $(1, 2)$.

```

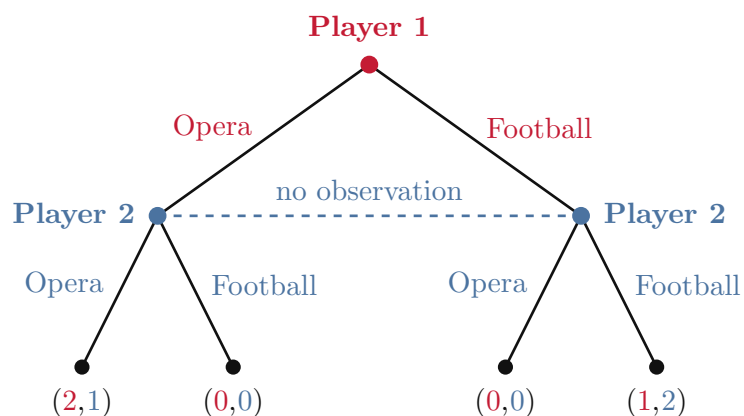
1  #cetz.canvas({
2      import cetz.draw: *
3      game-branch((0,0), (-2.8,-2),
4          action: [Opera],
5          player: 1,
6          side: "w")
7      game-branch((0,0), ( 2.8,-2),

```

```

8      action: [Football],
9      player: 1,
10     side: "e")
11  game-infoset((-2.8,-2), (2.8,-2),
12      player: 2,
13      style: "dashed",
14      label: [no observation],
15      la: "n")
16  game-branch((-2.8,-2), (-3.8,-4),
17      action: [Opera],
18      player: 2,
19      side: "w")
20  game-terminal((-3.8,-4), payoffs: ([2], [1]))
21  game-branch((-2.8,-2), (-1.8,-4),
22      action: [Football],
23      player: 2,
24      side: "e")
25  game-terminal((-1.8,-4), payoffs: ([0], [0]))
26  game-branch(( 2.8,-2), ( 1.8,-4),
27      action: [Opera],
28      player: 2,
29      side: "w")
30  game-terminal(( 1.8,-4), payoffs: ([0], [0]))
31  game-branch(( 2.8,-2), ( 3.8,-4),
32      action: [Football],
33      player: 2,
34      side: "e")
35  game-terminal(( 3.8,-4), payoffs: ([1], [2]))
36  game-node((0, 0), player: 1, label: [Player 1], la: "n")
37  game-node((-2.8,-2), player: 2, label: [Player 2], la: "w")
38  game-node(( 2.8,-2), player: 2, label: [Player 2], la: "e")
39 })

```



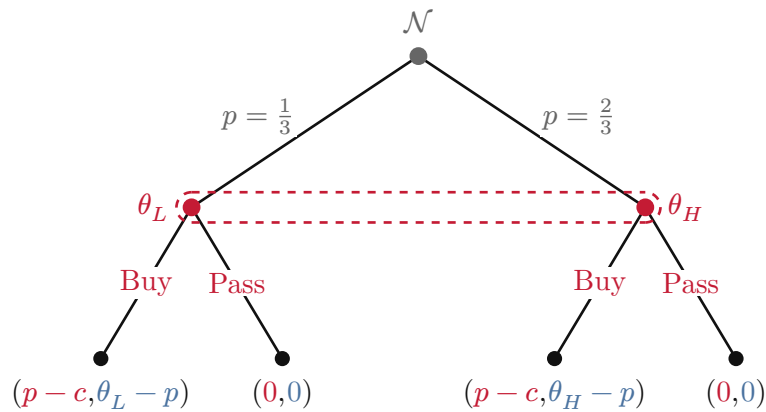
8.3 Adverse Selection with Nature

Nature draws a buyer's valuation: *Low* (θ_L) with probability $\frac{1}{3}$ and *High* (θ_H) with probability $\frac{2}{3}$. Only the buyer observes her type; the seller offers a price p without knowing the realised type. The seller's ignorance is modelled by a bracket-style information set spanning the two buyer nodes.

```

1  #cetz.canvas({
2      import cetz.draw: *
3      game-prob((0,0), (-3,-2), action: [$p = 1/3$], side: "w")
4      game-prob((0,0), ( 3,-2), action: [$p = 2/3$], side: "e")
5      game-branch((-3,-2), (-4.2,-4), action: [Buy], player: 1)
6      game-terminal((-4.2,-4), payoffs: ([$p - c$], [$theta_L - p$]))
7      game-branch((-3,-2), (-1.8,-4), action: [Pass], player: 1)
8      game-terminal((-1.8,-4), payoffs: ([0], [0]))
9      game-branch(( 3,-2), ( 1.8,-4), action: [Buy], player: 1)
10     game-terminal(( 1.8,-4), payoffs: ([$p - c$], [$theta_H - p$]))
11     game-branch(( 3,-2), ( 4.2,-4), action: [Pass], player: 1)
12     game-terminal(( 4.2,-4), payoffs: ([0], [0]))
13     game-infoset((-3,-2), (3,-2), player: 1, style: "bracket")
14     game-nature((0,0), label: [$cal(N)$], la: "n")
15     game-node((-3,-2), player: 1, label: [$theta_L$], la: "w")
16     game-node(( 3,-2), player: 1, label: [$theta_H$], la: "e")
17 })

```



8.4 Beer–Quiche Signalling Game (Cho–Kreps 1987)

Nature draws the **Sender**'s type: *Strong* (S , probability 0.7) or *Weak* (W , probability 0.3). The Sender signals with *Beer* (B) or *Quiche* (Q). The **Receiver** observes the signal but not the type, then chooses *Duel* (D) or *No Duel* (N). Two information sets — one bent downward (angle: -33deg, bracket style) and one bent upward (angle: 33deg, dashed style) — connect nodes reached by the same signal. The unique pooling Perfect Bayesian Equilibrium has both types choose Beer; the Receiver duels after Quiche (off-path belief: Weak). PBNE payoffs: (3, 1) for Strong, (2, 0) for Weak.

```

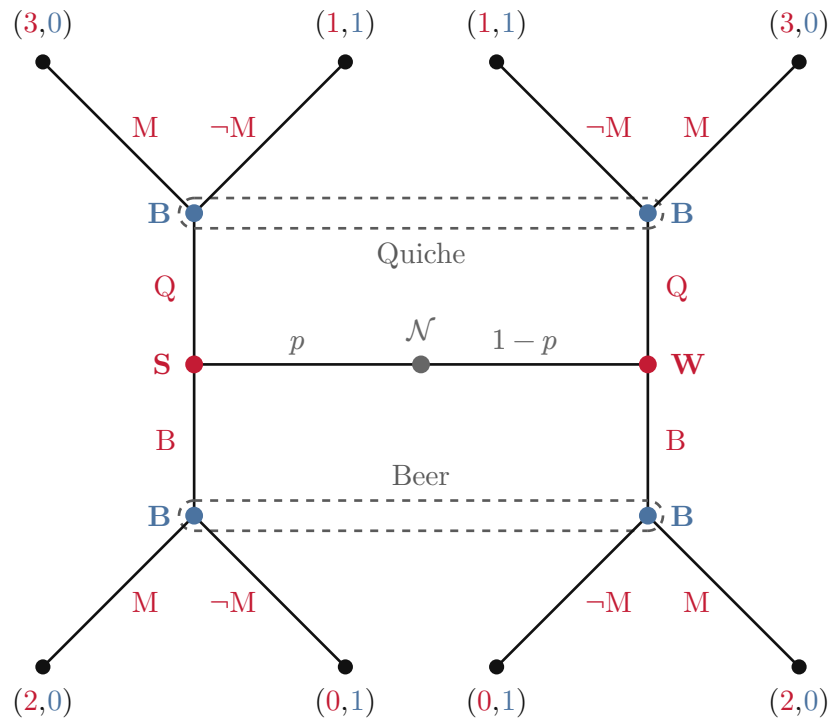
1  #cetz.canvas(
2      {
3      import cetz.draw: *
4      game-prob((0,0),(-3,0),action: [$p$],
5          side: "w",
6          act: 0.3)
7      game-prob((0,0),(3,0), action: [$1-p$],
8          side: "e",
9          act: 0.3,
10         apos: 0.3)
11     game-branch((-3,0),(-3,2), action: [Q], side: "w")
12     game-branch((-3,0),(-3,-2), action: [B], side: "w")
13     game-branch((3,0),(3,2), action: [Q], side: "e")
14     game-branch((3,0),(3,-2), action: [B], side: "e")
15     game-infoset((-3,2),(3,2), style: "bracket",
16         label: [Quiche])
17     game-infoset((-3,-2),(3,-2), style: "bracket",
18         label: [Beer],
19         la: "n")
20     game-branch((-3,2),(-5,4), action: [M], side: "e")
21     game-terminal((-5,4), payoffs: ([3],[0]), la: "n")
22     game-branch((-3,2),(-1,4), action: [$not$M], side: "w")
23     game-terminal((-1,4), payoffs: ([1],[1]), la: "n")
24     game-branch((-3,-2),(-5,-4), action: [M], side: "e")
25     game-terminal((-5,-4), payoffs: ([2],[0]), la: "s")
26     game-branch((-3,-2),(-1,-4), action: [$not$M], side: "w")
27     game-terminal((-1,-4), payoffs: ([0],[1]), la: "s")
28     game-branch((3,2),(5,4), action: [M], side: "w")
29     game-terminal((5,4), payoffs: ([3],[0]), la: "n")
30     game-branch((3,2),(1,4), action: [$not$M], side: "e")
31     game-terminal((1,4), payoffs: ([1],[1]), la: "n")
32     game-branch((3,-2),(5,-4), action: [M], side: "w")
33     game-terminal((5,-4), payoffs: ([2],[0]), la: "s")
34     game-branch((3,-2),(1,-4), action: [$not$M], side: "e")
35     game-terminal((1,-4), payoffs: ([0],[1]), la: "s")
36     game-nature((0,0), label: [$cal(N)$], la: "n")
37     game-node((-3,0), label: [S], la: "w")
38     game-node((3,0), label: [W], la: "e")
39     game-node((-3,2), label: [B], player: 2, la: "w")
40     game-node((-3,-2), label: [B], player: 2, la: "w")
41     game-node((3,2), label: [B], player: 2, la: "e")
42     game-node((3,-2), label: [B], player: 2, la: "e")

```

```

43     }
44 )

```



9 Annotations

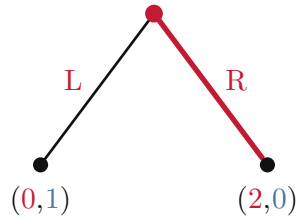
9.1 Equilibrium Highlights: **game-highlight**

`game-highlight` draws a bold coloured line over an existing branch to mark the equilibrium path. It is shortened by `game-node-radius` at each end so the highlight does not overlap the node circles. Call it *after* the branches but *before* the nodes so the nodes remain on top.

```

1  #cetz.canvas({
2    import cetz.draw: *
3    game-branch((0,0), (-1.5,-2), action: [L], player: 1, side: "w")
4    game-branch((0,0), ( 1.5,-2), action: [R], player: 1, side: "e")
5    // Highlight the right branch in player 1's colour
6    game-highlight((0,0), (1.5,-2), color: game-pal.at(0))
7    game-node((0,0), player: 1)
8    game-terminal((-1.5,-2), payoffs: ([0],[1]))
9    game-terminal(( 1.5,-2), payoffs: ([2],[0]))
10 })

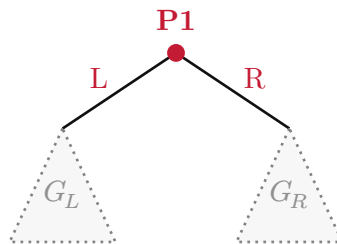
```



9.2 Proper-Subgame Markers: `game-subgame`

`game-subgame` draws a dotted, lightly shaded triangle with its apex at the subgame root. `depth` controls the height of the triangle and width its half-base width. An optional `label` is placed inside at roughly 42% of the height from the base.

```
1 #cetz.canvas({
2   import cetz.draw: *
3   game-branch((0,0), (-1.5,-1), action: [L], player: 1, side: "w")
4   game-branch((0,0), ( 1.5,-1), action: [R], player: 1, side: "e")
5   game-subgame((-1.5,-1), depth: 1.5, width: 0.7, label: [$G_L$])
6   game-subgame(( 1.5,-1), depth: 1.5, width: 0.7, label: [$G_R$])
7   game-node((0,0), player: 1, label: [P1], la: "n")
8 })
```



10 Inline Text Helpers

10.1 Coloured Payoff Vectors: `game-payoffs`

`game-payoffs` typesets a payoff vector in body text or captions, colouring each entry in that player's colour. It returns ordinary Typst content and can be used anywhere — inside and outside `cetz.canvas` alike.

```
1 The unique NE payoff vector is #game-payoffs(([2], [1])).
2
3 In math:  $u = \text{\textcolor{blue}{a}} - \text{\textcolor{red}{c}}, \text{\textcolor{teal}{b}}.$ 
```

The unique NE payoff vector is (2, 1).

In math: $u = \text{\textcolor{red}{a}} - \text{\textcolor{red}{c}}, \text{\textcolor{teal}{b}}.$

10.2 Player Names in Colour: `game-player` and `game-player-default`

`game-player(n)[label]` typesets arbitrary content in player n 's colour (bold). `game-player-default(n)` is a shortcut that produces “Player n ” without requiring a label argument.

```

1 #game-player(1)[Challenger] moves first;
2 #game-player(2)[Incumbent] responds.
3 Payoffs: #game-payoffs(([1], [1])).
4 Both players are #game-player-default(1) and #game-player-default(2).

```

Challenger moves first; **Incumbent** responds. Payoffs: (1, 1). Both players are **Player 1** and **Player 2**.

11 Configuration — Extensive-Form Trees

All geometric and typographic constants are ordinary `let` bindings. Override them by redefining the binding *after* importing the library:

```

1 #import "pi-games.typ": *
2 #let game-node-radius      = 0.15    // larger decision nodes
3 #let game-terminal-radius  = 0.08    // smaller terminal dots
4 #let game-act              = 0.3     // more space for action labels
5 #let game-fsl              = 10pt    // smaller player-label font
6 #let game-highlight-color = rgb("#e53e3e") // custom highlight colour
7 #let game-pal = (           // custom player colours
8   rgb("#005f73"), rgb("#94d2bd"), rgb("#e9d8a6"),
9   rgb("#ee9b00"), rgb("#ae2012"),
10 )

```

The full list of configuration bindings and their defaults is given in the API Reference below.

Name	Default	Role
game-pal	5-colour list	Player colours (index 0 = Player 1).
game-nature-color	rgb("#666666")	Nature / chance node colour.
game-fg	rgb("#111111")	Branch lines, terminal dots, payoff punctuation.
game-highlight-color	rgb("#e53e3e")	Default game-highlight overlay colour.
game-info-set-color	luma(90)	Information-set colour when no player: is given.
game-subgame-stroke	luma(130)	Subgame triangle outline colour.
game-subgame-fill	luma(247)	Subgame triangle fill colour.
game-subgame-label	luma(145)	Subgame triangle label colour.
game-node-radius	0.1 cm	Decision node radius.
game-terminal-radius	0.1 cm	Terminal dot radius.
game-gap	0.2 cm	Node edge to label gap.
game-act	0.2 cm	Branch-line to action-label offset.
game-apos	0.5	Fractional label position along branch.

Name	Default	Role
game-tick	0.20 cm	Bracket tick half-length.
game-sw-b	1pt	Branch stroke width.
game-sw-n	1pt	Decision-node stroke width.
game-sw-ni	1pt	Nature-node stroke width.
game-sw-i	1pt	Information-set stroke width.
game-sw-h	2pt	Highlight stroke width.
game-fsl	11pt	Player / node label font size.
game-fsa	11pt	Action label font size.
game-fsp	11pt	Payoff label font size.

API Reference

pi-game-normal.typ

- game-normal-form()
- game-three-player-normal-form()

Variables

- game-pal
- game-nash-color
- game-fg
- game-cell-width
- game-cell-height
- game-games-per-row

game-normal-form

Draws a general $N \times M$ normal form game table with colored payoffs, best-response underlines, and Nash equilibrium highlights.

Parameters

```
game-normal-form(
  p1: content,
  p2: content,
  s1: array,
  s2: array,
  payoffs,
  p1-best,
  p2-best,
  nash,
  cell-width: length,
  cell-height: length
) -> content
```

p1 content

Name of the row player, displayed rotated 90° on the left side of the matrix. Accepts any Typst content, e.g. [Alice] or [\$P_1\$].

p2 content

Name of the column player, displayed in bold above the column strategy labels. Accepts any Typst content.

s1 array

Array of strategy labels for Player 1, one element per row. The number of rows equals `s1.len()`. Each label is content and may include math or line breaks (e.g. [\$p\$\\ C] for a two-line label).

s2 array

Array of strategy labels for Player 2, one element per column. The number of columns equals `s2.len()`. Column width auto-adjusts to the widest label in this array.

payoffs

2D array of payoff pairs indexed as `payoffs.at(row).at(col) = (v1, v2)`. Both `v1` (Player 1) and `v2` (Player 2) are Typst content and may contain arbitrary markup or mathematics. Example for a 2×2 game:

```
1 (
2   (([$3$],[ $3$]), ([$0$],[ $5$])),    // row 0
3   (([$5$],[ $0$]), ([$1$],[ $1$])),    // row 1
4 )
```

p1-best

Array of (row, col) tuples identifying cells where Player 1 plays a best response. Player 1's payoff in those cells is underlined in `game-pal.at(0)`.

Default: ()

p2-best

Array of (row, col) tuples identifying cells where Player 2 plays a best response. Player 2's payoff in those cells is underlined in `game-pal.at(1)`.

Default: ()

nash

Array of (row, col) tuples of Nash equilibrium cells. A rectangle in `game-nash-color` is drawn just inside the cell border.

Default: ()

cell-width `length`

Minimum cell width. The actual width is the maximum of this value, the widest Player 2 strategy label width plus 2em, and the widest payoff pair width plus 2em. Increase to enforce a larger minimum regardless of content. Defaults to `game-cell-width`.

Default: `game-cell-width`

cell-height `length`

Fixed cell height. Content is placed at the cell centre; this value is not auto-sized, so increase it when payoffs span multiple lines. Defaults to `game-cell-height`.

Default: `game-cell-height`

game-three-player-normal-form

Draws a three-player normal form game as a collection of $N \times M$ payoff matrices, one per strategy of Player 3, laid out in rows of at most `games-per-row` matrices. All sub-matrices share the same cell dimensions. Payoffs for all three players are shown in each cell. Strategy labels of Player 3 appear above each sub-matrix.

Parameters

```
game-three-player-normal-form(  
  p1: content,  
  p2: content,  
  p3: content,  
  s1: array,  
  s2: array,  
  s3: array,  
  payoffs,  
  p1-best,  
  p2-best,  
  p3-best,  
  nash,  
  cell-width: length,  
  cell-height: length,  
  games-per-row: int  
) -> content
```

p1 content

Name of the row player (Player 1), displayed rotated 90° to the left of each row of sub-matrices.

p2 content

Name of the column player (Player 2), displayed in bold above the strategy labels of each sub-matrix.

p3 content

Name of the matrix player (Player 3), displayed once in bold above the first row of sub-matrices.

s1 array

Array of strategy labels for Player 1, shared across all sub-matrices. Determines the number of rows; `s1.len()` rows are drawn.

s2 array

Array of strategy labels for Player 2, shared across all sub-matrices. Determines the number of columns; `s2.len()` columns are drawn.

s3 **array**

Array of strategy labels for Player 3. One sub-matrix is drawn per entry, so `s3.len()` sub-matrices are produced in total.

payoffs

3D array of payoff triples indexed as `payoffs.at(k).at(r).at(c) = (v1, v2, v3)`. `k` is the Player 3 strategy index (outermost), `r` the row (Player 1), `c` the column (Player 2). All three values are Typst content.

p1-best

Array of `(k, row, col)` tuples where Player 1 plays a best response. Player 1's payoff is underlined in `game-pal.at(0)` at those cells.

Default: `()`

p2-best

Array of `(k, row, col)` tuples where Player 2 plays a best response. Player 2's payoff is underlined in `game-pal.at(1)` at those cells.

Default: `()`

p3-best

Array of `(k, row, col)` tuples where Player 3 plays a best response. Player 3's payoff is underlined in `game-pal.at(2)` at those cells.

Default: `()`

nash

Array of `(k, row, col)` tuples of Nash equilibrium cells. A rectangle in `game-nash-color` is drawn inside those cells.

Default: `()`

cell-width **length**

Minimum cell width, auto-grown to fit the widest payoff triple or Player 2 strategy label across all sub-matrices. Defaults to `game-cell-width`.

Default: `game-cell-width`

cell-height `length`

Fixed cell height. Increase when payoff content spans multiple lines. Defaults to `game-cell-height`.

Default: `game-cell-height`

games-per-row `int`

Maximum number of sub-matrices placed side by side. If Player 3 has more strategies than this, additional rows of sub-matrices are added below. A lone sub-matrix in the last row is horizontally centred. Defaults to `game-games-per-row`.

Default: `game-games-per-row`

game-pal

Per-player colour palette. Index 0 = Player 1, index 1 = Player 2, index 2 = Player 3. Defaults to `game-player-colors` from `pi-game-palette`. Override after import: `#let game-pal = (rgb("..."), ...)`.

game-nash-color

Colour for Nash equilibrium cell outlines. Defaults to `game-nash-color` from `pi-game-palette`. Override after import: `#let game-nash-color = rgb("...")`.

game-fg

Foreground colour for cell borders, comma separators, and punctuation. Defaults to `game-fg` from `pi-game-palette`. Override after import: `#let game-fg = rgb("...")`.

game-cell-width

Minimum cell width for payoff matrix columns [length]. The actual width is auto-grown to fit the widest payoff or strategy label. Override after import: `#let game-cell-width = 6em`.

game-cell-height

Fixed cell height for payoff matrix rows [length]. Override after import: `#let game-cell-height = 3em`.

game-games-per-row

Maximum number of sub-matrices placed side by side in three-player games [int]. Override after import: `#let game-games-per-row = 3`.

pi-game-trees.typ

- `game-branch()`
- `game-highlight()`
- `game-infoset()`
- `game-nature()`
- `game-node()`
- `game-payoffs()`
- `game-player()`
- `game-player-default()`
- `game-prob()`
- `game-subgame()`
- `game-terminal()`

Variables

- `game-pal`
- `game-nature-color`
- `game-fg`
- `game-highlight-color`
- `game-infoset-color`
- `game-subgame-stroke`
- `game-subgame-fill`
- `game-subgame-label`
- `game-node-radius`
- `game-terminal-radius`
- `game-gap`
- `game-act`
- `game-apos`
- `game-tick`
- `game-sw-b`
- `game-sw-n`
- `game-sw-ni`
- `game-sw-i`
- `game-sw-h`

- game-fsl
- game-fsa
- game-fsp

game-branch

Draw a branch (edge) from a parent node to a child node, with an optional action label.

Parameters

```
game-branch(
    from: array,
    to: array,
    action: content,
    player: int,
    side: str,
    act: float,
    apos: float,
    arrow: bool
)
```

from array

Parent node centre (x, y).

to array

Child node centre (x, y).

action content

Action label text. Pass `none` for no label. Default: `none`.

Default: `none`

player int

Player whose colour is used for the label. 0 uses `game-nature-color`. Default: 1.

Default: `1`

side str

Side of the branch for the label. `none` — centred on the line with a white background (default); `"e"` — east (absolute) side, label anchored `"west"`; `"w"` — west (absolute) side, label anchored `"east"`.

Default: `none`

act `float`

Perpendicular distance in cm from the branch line to the label anchor. Effective only when `side` is "e" or "w". Default: `game-act`.

Default: `game-act`

apos `float`

Fractional position of the label along the branch. 0 = parent end, 1 = child end. Default: `game-apos`.

Default: `game-apos`

arrow `bool`

Draw an arrowhead at the child end of the branch. Default: `false`.

Default: `false`

game-highlight

Draw a bold coloured overlay on an existing branch to mark an equilibrium path.

Parameters

```
game-highlight(  
  from: array,  
  to: array,  
  color: color,  
  width: length  
)
```

from `array`

Start node centre (x, y).

to `array`

End node centre (x, y).

color `color`

Highlight colour. Default: `game-highlight-color`.

Default: `game-highlight-color`

width **length**

Stroke width. Default: `game-sw-h` (2pt).

Default: `game-sw-h`

game-infoset

Draw an information set connecting two or more decision nodes.

Parameters

```
game-infoset(  
  ..pts-arg: array,  
  player: int,  
  style: str,  
  angle: angle,  
  label: content,  
  la: str  
)
```

..pts-arg **array**

Node positions. Two-point form: two separate coordinate pairs (`x`, `y`). List form: one argument that is itself an array of pairs (`(x1,y1)`, `(x2,y2)`, ...).

player **int**

Player index for the colour. 1–5 use `game-pal`; `none` uses dark grey. Default: `none`.

Default: `none`

style **str**

Drawing style. "`dashed`" — dashed line(s) between consecutive points (default); "`bracket`" — rounded bracket ribbon with half-circle caps at each endpoint.

Default: "`dashed`"

angle `angle`

Segment curvature. `none` — straight (default); a single angle — same bend for every segment; an array of angles — one per segment (length = number of points — 1). Positive angle bends toward the CCW perpendicular. For "bracket" only the first entry is used.

Default: `none`

label `content`

Label at the midpoint of the first-to-last span. For "dashed" style, only shown when exactly two points are given. Default: `none`.

Default: `none`

la `str`

Direction for the label offset. One of "n", "s", "e", "w", "ne", "nw", "se", "sw". Default: "s".

Default: `"s"`

game-nature

Draw a Nature / chance node (grey filled circle).

Parameters

```
game-nature(  
  pos: array,  
  label: content,  
  la: str,  
  name: str  
)
```

pos `array`

Node centre as a CeTZ coordinate pair (x, y).

label `content`

Label placed near the node. Pass `none` to suppress. Default: `$N$`.

Default: `[$N$]`

la `str`

Direction for the label offset from the node centre. One of "n", "s", "e", "w", "ne", "nw", "se", "sw". Default: "n".

Default: `"n"`

name `str`

CeTZ coordinate name for this node. Pass `none` to skip. Default: `none`.

Default: `none`

game-node

Draw a decision (non-terminal) node for a given player.

Parameters

```
game-node(  
    pos: array,  
    player: int,  
    label: content,  
    la: str,  
    name: str,  
    style: str  
)
```

pos `array`

Node centre as a CeTZ coordinate pair (x, y).

player `int`

Player index. 1–5 use the corresponding colour from `game-pal`; 0 uses `game-nature-color` (grey). Default: 1.

Default: `1`

label `content`

Player name or label placed near the node. Pass `none` to suppress.

Default: `none`

la `str`

Direction for the label offset from the node centre. One of "n", "s", "e", "w", "ne", "nw", "se", "sw".

Default: "n"

name `str`

CeTZ coordinate name assigned to this node for later reference (e.g. in `game-info`set). Pass `none` to skip.

Default: `none`

style `str`

Visual style of the node circle. "filled" — solid filled circle (default); "open" — hollow circle with coloured outline; "dot" — small solid disc for compact trees.

Default: "filled"

game-payoffs

Build a coloured inline payoff vector for body text or math. Each payoff is coloured by player.

Parameters

```
game-payoffs(  
    payoffs: array,  
    parens: bool  
) -> content
```

payoffs `array`

Array of content, one element per player, e.g. ([2], [1]) or ($[p-c]$, $[v-p]$).

parens `bool`

Wrap the payoff vector in parentheses. Default: `true`.

Default: `true`

game-player

Typeset arbitrary content in a player's colour (bold).

Parameters

```
game-player(  
  player: int,  
  label: content  
) -> content
```

player `int`

Player index. 1–5 use game-pal; 0 uses game-nature-color.

label `content`

Text to render in the player's colour.

game-player-default

Typeset the default label “Player N” in player N's colour (bold).

Parameters

```
game-player-default(player: int) -> content
```

player `int`

Player index. 1–5 use game-pal; 0 uses game-nature-color.

game-prob

Place a probability label on a Nature branch. Wrapper around game-branch with player: 0.

Parameters

```
game-prob(  
  from: array,  
  to: array,  
  action: content,  
  side: str,  
  act: float,  
  apos: float,  
  arrow: bool  
)
```

from `array`

Parent node centre (x, y).

to `array`

Child node centre (x, y).

action `content`

Probability expression, e.g. [\$p\$]. Default: none.

Default: `none`

side `str`

Label placement. `none` — on the branch line (default); "e" — east side; "w" — west side. See `game-branch`.

Default: `none`

act `float`

Perpendicular offset in cm from the branch line. Default: `game-act`.

Default: `game-act`

apos `float`

Fractional position of the label along the branch. Default: `game-apos`.

Default: `game-apos`

arrow `bool`

Draw arrowhead at the child end. Default: `false`.

Default: `false`

game-subgame

Draw a proper-subgame triangle marker (dotted, lightly filled, apex at the root).

Parameters

```
game-subgame(  
    apex: array,  
    depth: float,  
    width: float,  
    label: content  
)
```

apex array

Position of the subgame root node (x, y).

depth float

Height of the triangle downward in cm. Default: 1.0.

Default: 1.0

width float

Half-width of the base in cm. Default: 0.6.

Default: 0.6

label content

Label inside the triangle at 42% of the height from the base. Pass none to suppress.
Default: none.

Default: none

game-terminal

Draw a terminal node (small filled dot) with a coloured payoff vector.

Parameters

```
game-terminal(  
    pos: array,  
    payoffs: array,  
    la: str,  
    parens: bool  
)
```

pos `array`

Node centre as a CeTZ coordinate pair (x, y).

payoffs `array`

Array of content, one element per player, e.g. ([2], [3]) or ([\$p-c\$], [\$v-p\$]).
Pass `none` or an empty array to draw only the dot. Default: `none`.

Default: `none`

la `str`

Direction for the payoff label. One of "n", "s", "e", "w", "ne", "nw", "se", "sw".
Default: "s".

Default: `"s"`

parens `bool`

Wrap the payoff vector in parentheses. Default: `true`.

Default: `true`

game-pal

Per-player colour palette. Index 0 = Player 1, index 4 = Player 5. Defaults to `game-player-colors` from `pi-game-palette`. Override after import: `#let game-pal = (rgb("..."), ...)`.

game-nature-color

Colour for Nature / chance nodes and probability labels. Defaults to `game-nature-color` from `pi-game-palette`. Override after import: `#let game-nature-color = rgb("...")`.

game-fg

Foreground colour for branches, terminal dots, and payoff punctuation. Defaults to `game-fg` from `pi-game-palette`. Override after import: `#let game-fg = rgb("...")`.

game-highlight-color

Default colour for equilibrium-path highlight overlays drawn by `game-highlight`. Defaults to `game-highlight-color` from `pi-game-palette`. Override after import: `#let game-highlight-color = rgb("...")`.

game-infoset-color

Fallback stroke colour for information-set lines when no `player` is given to `game-infoset`. Defaults to `game-infoset-color` from `pi-game-palette`. Override after import: `#let game-infoset-color = luma(90)`.

game-subgame-stroke

Stroke colour for proper-subgame triangle outlines drawn by `game-subgame`. Defaults to `game-subgame-stroke` from `pi-game-palette`. Override after import: `#let game-subgame-stroke = luma(130)`.

game-subgame-fill

Fill colour for proper-subgame triangle interiors drawn by `game-subgame`. Defaults to `game-subgame-fill` from `pi-game-palette`. Override after import: `#let game-subgame-fill = luma(247)`.

game-subgame-label

Label colour inside proper-subgame triangles drawn by `game-subgame`. Defaults to `game-subgame-label` from `pi-game-palette`. Override after import: `#let game-subgame-label = luma(145)`.

game-node-radius

Radius of decision nodes [cm].

game-terminal-radius

Radius of terminal filled-dot nodes [cm].

game-gap

Gap between node edge and adjacent player/payoff label [cm].

game-act

Default perpendicular distance from a branch line to an action label [cm].

game-apos

Default fractional position of an action label along a branch (0 = parent, 1 = child).

game-tick

Half-length of the bracket tick marks at information-set endpoints [cm].

game-sw-b

Stroke width for ordinary branches.

game-sw-n

Stroke width for decision-node outlines.

game-sw-ni

Stroke width for nature-node outlines.

game-sw-i

Stroke width for information-set lines.

game-sw-h

Stroke width for highlight overlays.

game-fsl

Font size for player and node labels.

game-fsa

Font size for action labels.

game-fsp

Font size for payoff labels.